COMP4620/8620 Advanced Topics in AI: Intelligent Robotics Assignment 2 Report

Hadis Amin - u8050066

Part A

A1. State abstraction of the 1-D grid MDP

No. Abstracting the state into L and R does not turn this into an MDP. An MDP assumes the agent knows exactly which state it is in. Here the true state is still uncertain: we only believe the agent is somewhere on the left side or the right side, not exactly where. The agent therefore still needs a belief, a probability distribution over the possible states, so the problem remains a POMDP.

A2. α-vector policy representation

Let $b = (p, 1-p)$ where $p = b(s_1) \in [0, 1]$. Then $\alpha \cdot b = \alpha(s_1)\, p + \alpha(s_2)\,(1-p)$.

α	vector	$V_\alpha(p) = \alpha \cdot b$
α_1	$[10, -5]$	$15p - 5$
α_2	$[-5, 10]$	$-15p + 10$
α_3	$[4, 4]$	4
α_4	$[1, 2]$	$-p + 2$
α_5	$[5.5, 1]$	$4.5p + 1$

Table 1: Value of each α-vector as a function of $p = b(s_1)$.

(a) Belief intervals where each α-vector is maximal. Figure 1 plots the five lines over $p \in [0, 1]$. The shaded upper surface is made of three segments: α_2 on the left, α_3 in the middle and α_1 on the right. The breakpoints (marked in the figure) are the intersections $\alpha_2 = \alpha_3 \Rightarrow -15p + 10 = 4 \Rightarrow p = 0.4$ and $\alpha_3 = \alpha_1 \Rightarrow 15p - 5 = 4 \Rightarrow p = 0.6$. The figure also shows that α_4 and α_5 lie strictly below the surface everywhere: α_4 never exceeds $2 < 4$, and α_5 is below α_3 on $[0, 2/3)$ and below α_1 for $p > 4/7$, so neither is ever maximal. The resulting intervals are summarised in Table 2.

α	Interval of $p = b(s_1)$ where maximal
α_1	[0.6, 1]
α_2	[0, 0.4]
α_3	[0.4, 0.6]
α_4	never
α_5	never

Table 2: Belief intervals where each α-vector is maximal (read off Figure 1).

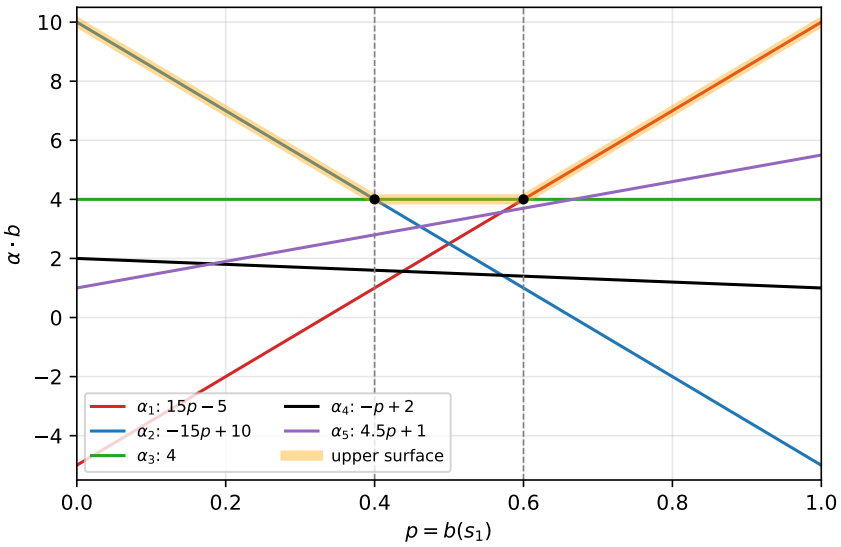


Figure 1: The five α-vectors over $p \in [0, 1]$ and the upper surface of Γ (shaded), with breakpoints at $p = 0.4$ and $p = 0.6$.

(b) Pruning by pointwise dominance. From Figure 1, α_4 and α_5 lie strictly below the maximum for every belief $p \in [0, 1]$, so both can be removed by pointwise dominance. The maximum is

$$V_{\max}(p) = \begin{cases} V_{\alpha_2}(p) = 10 - 15p, & 0 \le p \le 0.4, \\ V_{\alpha_3}(p) = 4, & 0.4 < p < 0.6, \\ V_{\alpha_1}(p) = 15p - 5, & 0.6 \le p \le 1. \end{cases}$$

For all $p \in [0, 1]$, $V_{\alpha_4}(p) = 2 - p < V_{\max}(p)$ and $V_{\alpha_5}(p) = 4.5p + 1 < V_{\max}(p)$. The dominating vector may differ with p. In contrast, α_1, α_2 and α_3 each attain the maximum on an interval, so they cannot be removed.

A3. Pedestrian avoidance: MDP vs POMDP

Scenario where MDP is suitable.

- S: [car_position, pedestrian_position]
- A: [Stay, Move]

Sensors accurately identify both positions. Assume movement is discrete and the next state depends only on the current positions and action. An MDP policy maps the observed state to the best action: if a pedestrian blocks the cell ahead, choose Stay; otherwise, choose Move when safe. Since the complete state is observed, a POMDP belief is unnecessary.

Scenario where POMDP is suitable. **[TODO: Describe the scenario, e.g. what part of the state the car cannot observe even with good sensors (pedestrian intention/destination?).]**

- State (observed and hidden parts): **[TODO:]**
- Observations: **[TODO:]**
- Why reasoning over a belief changes the decision: **[TODO:]**

Part B

B1. MDP framing of multi-drone navigation

- **State space** S: each state contains the positions of all N drones, $s = (p_1, \ldots, p_N)$, where $p_i = (x_i, y_i, z_i)$.

- **Action space** A: A joint action $a = (a_1, \ldots, a_N)$ specifies the simultaneous action of every drone. Thus, $A = A_1 \times \cdots \times A_N$ and $|A| = 26^N$. For example, two drones have $26^2 = 676$ joint actions.
- **Transition function** $T(s, a, s') = T_{known}$
- **Reward function** for N drones is the sum over all drones:

$$R(s, a, s') = \sum_{i=1}^{N} R_i(s, a, s'),$$

where

$$R_i(s, a, s') = -1 + \begin{cases} -50, & \text{if drone } i \text{ collides,} \\ 100, & \text{if drone } i \text{ reaches its goal.} \end{cases}$$

- **Discount factor** γ: **[TODO: e.g. 0.99]**

B2. Approximate on-line MDP solver

Overview. We use Monte Carlo Tree Search (MCTS) with UCT selection and a greedy rollout policy. At each decision step, the planner builds a search tree rooted at the current joint state within the available planning time (at most 10 seconds, without parallelisation). It estimates action values through sampled transitions and discounted returns, then executes the action with the highest estimated value at the root. Sampling avoids enumerating all possible successor states, although a large joint action space remains a limitation.

Component specification

- **Selection:** Randomly select an untried action first. Once every action has been tried, use UCB to select $a \leftarrow \arg\max_a \left[Q(s, a) + c\sqrt{\ln N(s)/N(s, a)}\right]$.
- **Expansion:** Add one new child for the sampled successor state and initialise its visit counts and action values to zero.
- **Simulation:** Use a greedy rollout: each drone moves closest to its goal. Estimate the new child's value by the discounted return $\hat{V} = \sum_{t=0}^{T-1} \gamma^t r_t$, stopping at a terminal state or the rollout depth limit.
- **Backpropagation:** Back up the discounted return to the root, updating the mean action value $Q(s, a)$ and incrementing the visit counts at each step.

Pseudocode

Let A be the set of joint-action integers, γ the configured discount factor, H the search depth limit (the configured maximum number of steps), and $c > 0$ the UCB exploration constant. Each node v stores its joint state $s(v)$, action counts $N(v, a)$, mean returns $Q(v, a)$, and successor nodes $C(v, a, s')$; its visit count is $N(v) = \sum_a N(v, a)$. A new node has zero counts and values, and no children; untried actions are represented implicitly. Each iteration descends the tree iteratively, records the visited (v, a, r) triples in a path, and backs up the return along that path in reverse. All functions share the planning deadline.

Algorithm 1: Online MCTS with UCT selection

Function `Plan`(s_0, *timeBudget*)**:**
  $root \leftarrow$ `NewNode`(s_0);
  $deadline \leftarrow now() + timeBudget$;
  while $now() < deadline$ **do**
    $path \leftarrow [\,]$; $v \leftarrow root$; $d \leftarrow H$; $G \leftarrow 0$;
    `// Selection, expansion and simulation`
    while $d > 0$ *and* $now() < deadline$ **do**
      $a \leftarrow$ `Select`(v);
      $(s', r, done, info) \leftarrow env.simulate(s(v), a)$;
      append (v, a, r) to *path*; $d \leftarrow d - 1$;
      if *done* **then**
        break;
      if $C(v, a, s')$ *does not exist* **then**
        $C(v, a, s') \leftarrow$ `NewNode`(s');
        $G \leftarrow$ `Rollout`(s', d);
        break;
      $v \leftarrow C(v, a, s')$;
    `// Backpropagation`
    foreach (v, a, r) *in path, in reverse order* **do**
      $G \leftarrow r + \gamma G$;
      $N(v, a) \leftarrow N(v, a) + 1$;
      $Q(v, a) \leftarrow Q(v, a) + \frac{G - Q(v,a)}{N(v,a)}$;
  if *no action has been tried at root* **then**
    return `GreedyJointAction`(s_0);
  return $\arg\max_{a:\, N(root,a)>0} Q(root, a)$;

Function `Select`(v)**:**
  if *there exists* $a \in A$ *with* $N(v, a) = 0$ **then**
    return a uniformly sampled untried joint action;
  return $\arg\max_{a \in A} \left[Q(v, a) + c\sqrt{\frac{\ln N(v)}{N(v,a)}} \right]$;

Algorithm 2: Greedy rollout and joint-action construction

Function `Rollout`(s, d)**:**
  $G \leftarrow 0$; $w \leftarrow 1$;
  for $t \leftarrow 1$ **to** d **do**
    if $now() \geq deadline$ **then**
      break;
    $a \leftarrow$ `GreedyJointAction`(s);
    $(s', r, done, info) \leftarrow env.simulate(s, a)$;
    $G \leftarrow G + w\,r$;
    if *done* **then**
      break;
    $s \leftarrow s'$; $w \leftarrow w\gamma$;
  return G;

Function `GreedyJointAction`(s)**:**
  for *each drone* i **do**
    if *drone* i *has already reached its goal* **then**
      $a_i \leftarrow 0$; `// The simulator keeps it at its goal.`
    else
      $F_i \leftarrow$ moves with intended destinations inside free grid cells;
      if $F_i \neq \varnothing$ **then**
        $a_i \leftarrow \arg\min_{u \in F_i} \|p_i + \Delta(u) - g_i\|_2$;
        Break ties uniformly at random;
      else
        $a_i \leftarrow$ uniformly sample a per-drone action;
  return $\sum_{i=1}^{N} a_i m^{i-1}$, where m is the number of per-drone actions;

Here p_i and g_i are drone i's current and goal positions, and $\Delta(u)$ is the displacement of per-drone action u. Action indices run from 0 to $m-1$, where $m = 8$ without altitude changes and $m = 26$ otherwise. The grid size, obstacle cells, goal positions and per-drone moves are read from the environment configuration (`get_config()`), so the planner only uses `simulate`, `get_config` and `num_actions`. The greedy policy evaluates intended destinations only; the simulator samples actual motion and detects both obstacle and inter-drone collisions. Each simulated transition uses the simulator's reward and terminal signal. Since transitions are stochastic, the same action can have multiple successor nodes. When the depth limit or deadline is reached, the remaining return is approximated as zero. The main execution loop applies the selected action using `env.step`, observes the next joint state, and calls `Plan()` again.

B4. Empirical investigation of problem complexity

Experimental setup

- Hardware / Python version: **[TODO:]**
- Planning time per step: **[TODO:]**; max steps: **[TODO:]**; α: **[TODO:]**; reward parameters: **[TODO:]**
- Runs per scenario: **[TODO: n]**; random seeds: **[TODO:]**
- 95% CI computed as $\bar{x} \pm t_{0.975,\,n-1}\, s/\sqrt{n}$ **[TODO: (or Wilson interval for success rate)]**.

Performance metrics.

- Success rate (all drones reach goal without collision)
- Total discounted reward
- Number of steps to completion (successful runs)
- Collision rate
- **[TODO: Tree statistics: simulations per step, average tree depth, etc.]**

Set	Varied factor (others fixed)	What we expect to learn
S: State space	**[TODO: grid size, e.g. $5^3, 10^3, 15^3, 20^3$; drones fixed]**	**[TODO: effect of longer horizons / sparser goal reward]**
A: Action space	**[TODO: number of drones N and/or altitude change (8 vs 26 per drone)]**	**[TODO: effect of branching factor on search breadth]**
C: Clutter	**[TODO: number/density of obstacles, narrow passages]**	**[TODO: effect on rollout quality and collision risk]**

Table 3: Experiment design. **[TODO: Note that increasing N also grows $|S|$; explain how the design separates the two factors.]**

Complexity factors and how each is varied (one factor at a time).

ID	Set	Grid	N	Altitude change	#Obstacles	$\|A\|$
S1	S	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
S2	S	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
S3	S	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
A1	A	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
A2	A	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
A3	A	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
C1	C	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
C2	C	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
C3	C	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**

Table 4: Testing scenarios. Config files in `[TODO: experiments/configs/]`.

Scenario list.

Results

ID	Success rate (95% CI)	Discounted reward (mean ± CI)	Steps (mean ± CI)	Sims/step (mean)
S1	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
S2	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
S3	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
A1	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
A2	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
A3	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
C1	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
C2	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
C3	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**

Table 5: Summary statistics over $n =$ **[TODO:]** runs per scenario.



(a) State space



(b) Action space



(c) Clutter

Figure 2: Mean discounted reward with 95% CI error bars as each factor increases.

Discussion

Effect of state-space size. **[TODO:]**

Effect of action-space size. **[TODO:]**

Effect of clutter. **[TODO:]**

Which factor degrades performance the most? **[TODO: Answer, supported by the statistics (overlapping vs non-overlapping CIs, normalised degradation, etc.).]**

Role of each solver component as complexity grows. **[TODO: Selection, expansion, rollout/heuristic, backpropagation, tree reuse: which helped or hurt, and why. Ablation results if run.]**

Limitations and possible improvements. **[TODO:]**

Appendix A: Generative AI disclosure

Tools used: **[TODO: tool names and versions, or "None"]**.
What they were used for: **[TODO:]**
Full list of prompts:

1. **[TODO: prompt 1 (verbatim)]**
2. **[TODO: prompt 2 (verbatim)]**

Appendix B: Additional results

[TODO: Optional: per-scenario raw tables, extra plots, ablations.]